

Normalization

Lecture 09

Borough of Manhattan Community College

The Purpose of Normalization

- ❑ **Normalization:** A technique for producing a set of relations with desirable properties, given the data requirements of an enterprise.
- ❑ The characteristics of the set of relations include the following:
 - the minimal number of attributes necessary to support the data requirements of the enterprise;
 - attributes with a close logical relationship are found in the same relation;
 - minimal redundancy, with each attribute represented only once, with the important exception of foreign keys, which are essential for the joining of related relations.
- ❑ The benefits are: The database will be easier for the user to access and maintain the data, and take up minimal storage space on the computer.

Data Redundancy and Update Anomalies

- ❑ Relations that have redundant data may have problems called **update anomalies**, which are classified as *insertion*, *modification*, and *deletion* anomalies.
- ❑ To illustrate the problems associated with unwanted data redundancy, we compare the **Staff** and **Branch** relations (*case 1*) with the alternative relation **StaffBranch** (*case 2*).
- ❑ The relations have the following form:

Staff (staffNo, sName, position, salary, branchNo)

Branch (branchNo, bAddress)

StaffBranch (staffNo, sName, position, salary, branchNo, bAddress)

Case 1

Staff

staffNo	sName	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SG5	Susan Brand	Manager	24000	B003
SL41	Julie Lee	Assistant	9000	B005

Branch

branchNo	bAddress
B005	22 Deer Rd, London
B007	16 Argyll St, Aberdeen
B003	163 Main St, Glasgow

Case 2

StaffBranch

staffNo	sName	position	salary	branchNo	bAddress
SL21	John White	Manager	30000	B005	22 Deer Rd, London
SG37	Ann Beech	Assistant	12000	B003	163 Main St, Glasgow
SG14	David Ford	Supervisor	18000	B003	163 Main St, Glasgow
SA9	Mary Howe	Assistant	9000	B007	16 Argyll St, Aberdeen
SG5	Susan Brand	Manager	24000	B003	163 Main St, Glasgow
SL41	Julie Lee	Assistant	9000	B005	22 Deer Rd, London

Insertion Anomalies

There are two main types of insertion anomaly:

❑ First type:

- To insert the details of new members of staff into the **StaffBranch** relation, we must include the correct details of the branch at which the staff are to be located. Otherwise the branch details won't be consistent for all staff working in the same branch.
- The relations **Staff** and **Branch** do not suffer from this potential inconsistency, because we enter only the appropriate branch number for each staff member in the **Staff** relation.

Insertion Anomalies

❑ Second type:

- To insert details of a new branch that currently has no members of staff into the **StaffBranch** relation, it is necessary to enter *nulls* into the attributes for staff, such as *staffNo*.

However, as *staffNo* is the primary key for the **StaffBranch** relation, attempting to enter *nulls* for *staffNo* violates entity integrity, and is not allowed. We therefore cannot enter a tuple for a new branch into the **StaffBranch** relation with a *null* for the *staffNo*.

- The design of the relations **Staff** and **Branch** avoids this problem, because branch details are entered in the **Branch** relation separately from the staff details. The details of staff ultimately located at that branch are entered at a later date into the **Staff** relation.

Modification Anomalies

- ❑ If we want to change the value of one of the attributes of a particular branch in the **StaffBranch** relation, we must update the tuples of all staff located at that branch.
- ❑ If this modification is not carried out on all the appropriate tuples of the **StaffBranch** relation, the database will become inconsistent (a branch may appear to have different addresses in different staff tuples).
- ❑ This demonstrates that although the **StaffBranch** relation is subject to update anomalies, we can avoid these anomalies by decomposing the original relation into the **Staff** and **Branch** relations.

Case 1

Staff

staffNo	sName	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SG5	Susan Brand	Manager	24000	B003
SL41	Julie Lee	Assistant	9000	B005

Branch

branchNo	bAddress
B005	22 Deer Rd, London
B007	16 Argyll St, Aberdeen
B003	163 Main St, Glasgow

Case 2

StaffBranch

staffNo	sName	position	salary	branchNo	bAddress
SL21	John White	Manager	30000	B005	22 Deer Rd, London
SG37	Ann Beech	Assistant	12000	B003	163 Main St, Glasgow
SG14	David Ford	Supervisor	18000	B003	163 Main St, Glasgow
SA9	Mary Howe	Assistant	9000	B007	16 Argyll St, Aberdeen
SG5	Susan Brand	Manager	24000	B003	163 Main St, Glasgow
SL41	Julie Lee	Assistant	9000	B005	22 Deer Rd, London

Deletion Anomalies

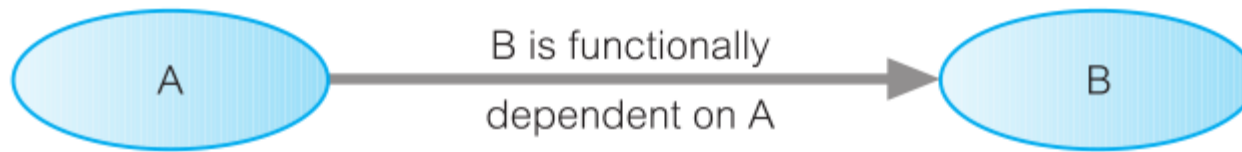
- ❑ If we delete a tuple from the **StaffBranch** relation that represents the last member of staff located at a branch, the details about that branch are also lost from the database.
 - For example, if we delete the tuple for staff number *SA9* (Mary Howe) from the **StaffBranch** relation, the details relating to branch number *B007* are lost from the database.
- ❑ The design of the relations **Staff** and **Branch** avoids this problem, because branch tuples are stored separately from staff tuples and only the attribute *branchNo* relates the two relations.
 - If we delete the tuple for staff number *SA9* from the **Staff** relation, the details on branch number *B007* remain unaffected in the **Branch** relation.

Functional Dependencies

- ❑ **Functional dependency:** Describes the relationship between attributes in a relation.
- ❑ For example, if A and B are attributes of relation R , B is functionally dependent on A (denoted $A \twoheadrightarrow B$), if each value of A is associated with exactly one value of B . (A and B may each consist of one or more attributes.)
- ❑ Consider a relation with attributes A and B , where attribute B is functionally dependent on attribute A . If we know the value of A and we examine the relation that holds this dependency, we find only one value of B in all the tuples that have a given value of A , at any moment in time.
- ❑ However, for a given value of B , there may be several different values of A .

Functional Dependencies

- The dependency between attributes A and B can be represented diagrammatically, as follows.

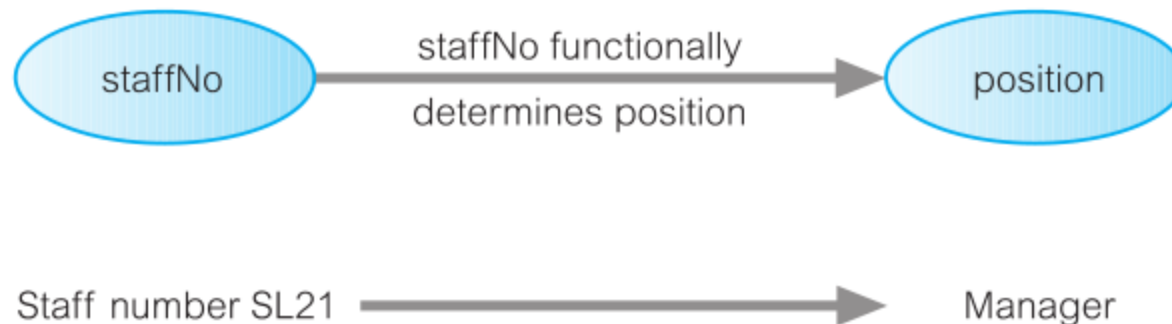


- An alternative way to describe the relationship between attributes A and B is to say that "A functionally determines B".
- **Determinant:** Refers to the attribute, or group of attributes, on the left-hand side of the arrow of a functional dependency.

Functional Dependencies - Example

- ❑ Consider the attributes **staffNo** and **position** of the **Staff** relation.
- ❑ For a specific **staffNo** - for example **SL21** - we can determine the position of that member of staff as **Manager**.

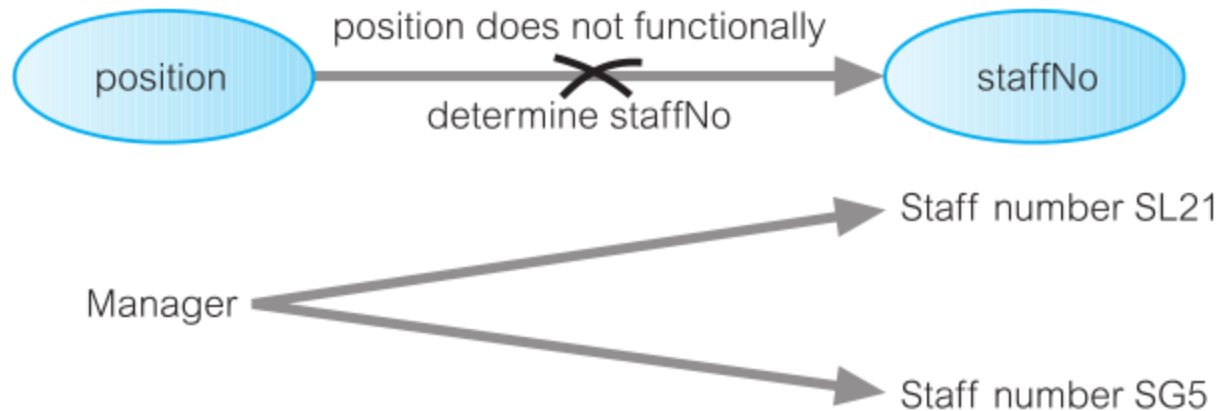
In other words, **staffNo** functionally determines **position**.



- ❑ In this example, **staffNo** is the determinant of this functional dependency.

Functional Dependencies - Example

- ❑ However the opposite is not true. A **position** does not functionally determine **staffNo**.
- ❑ A member of staff holds one position; however, there may be several members of staff with the same position.



Functional Dependencies

- ❑ For the purposes of normalization, we are interested in identifying functional dependencies between attributes of a relation.
- ❑ It is important to distinguish clearly between the values held by an attribute at a given point in time and the set of all possible values that an attribute may hold at different times.
- ❑ From the values shown in the **Staff** relation, can we conclude that the **sName** attribute functionally determines the **staffNo** attribute?
- ❑ Answer is NO as we may have two staff members with the same **sName** value in the future.

Functional Dependencies

- ❑ **Full functional dependency:** Indicates that if A and B are attributes of a relation, B is fully functionally dependent on A if B is functionally dependent on A, but not on any proper subset of A.
- ❑ A functional dependency $A \twoheadrightarrow B$ is a full functional dependency if removal of any attribute from A results in the dependency no longer existing.
- ❑ A functional dependency $A \twoheadrightarrow B$ is a **partial dependency** if there is some attribute that can be removed from A and yet the dependency still holds.
- ❑ **Transitive dependency:** A condition where A, B, and C are attributes of a relation such that if $A \twoheadrightarrow B$ and $B \twoheadrightarrow C$, then C is transitively dependent on A via B.

Functional Dependencies

- ❑ Consider the functional dependency that exists in the **Staff** relation:

$\text{staffNo, sName} \twoheadrightarrow \text{branchNo}$

- ❑ It is correct to say that each value of (**staffNo**, **sName**) is associated with a single value of **branchNo**. However, it is not a full functional dependency, because **branchNo** is also functionally dependent on a subset of (**staffNo**, **sName**), namely **staffNo**.
- ❑ In other words, the above functional dependency is an example of a partial dependency.
- ❑ The type of functional dependencies we are interested in is full functional dependency that hold for all time as shown here:

$\text{staffNo} \twoheadrightarrow \text{branchNo}$

Identifying Functional Dependencies

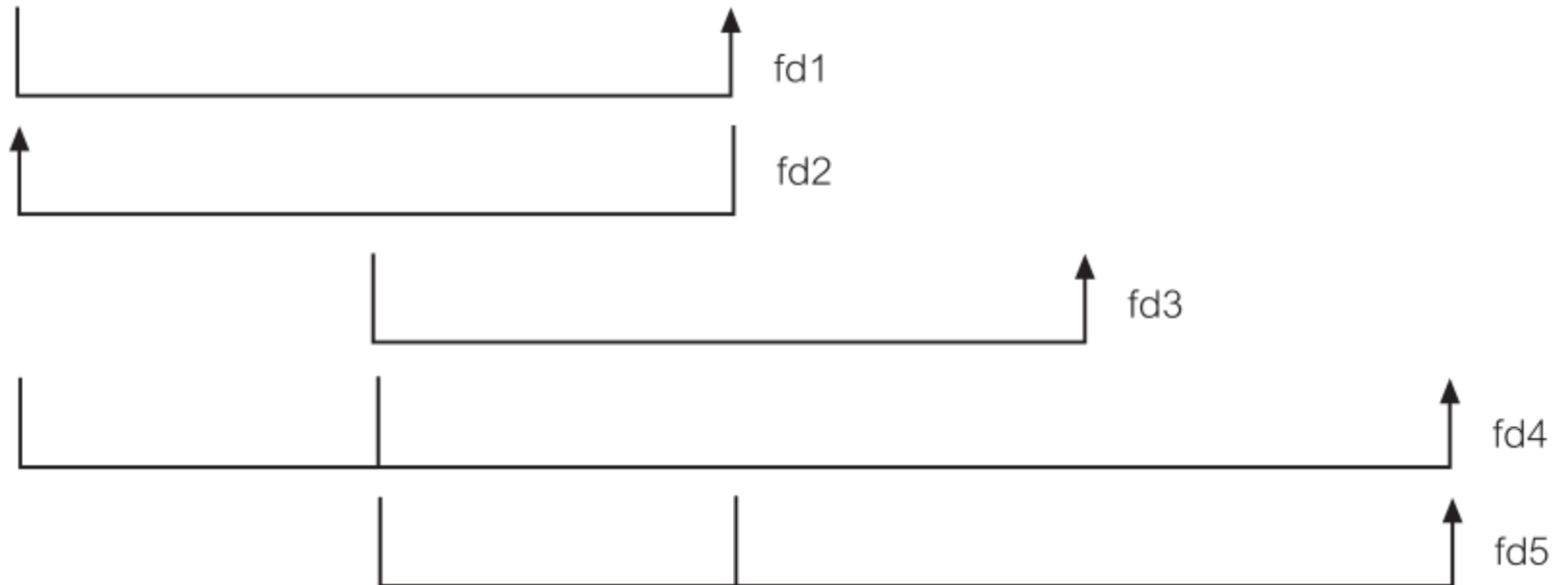
- ❑ Identifying all functional dependencies between a set of attributes should be quite simple if the meaning of each attribute and the relationships between the attributes are well understood.
- ❑ To identify the *FDs* in **StaffBranch** relation, we examine the semantics of its attributes. For the purposes of discussion, we assume that the position held and the branch determine a member of staff's salary.
- ❑ We identify the functional dependencies based on our understanding of the attributes in the relation as:
 - staffNo $\textcircled{R}$ sName, position, salary, branchNo, bAddress
 - branchNo $\textcircled{R}$ bAddress
 - bAddress $\textcircled{R}$ branchNo
 - branchNo, position $\textcircled{R}$ salary
 - bAddress, position $\textcircled{R}$ salary

Identifying Functional Dependencies

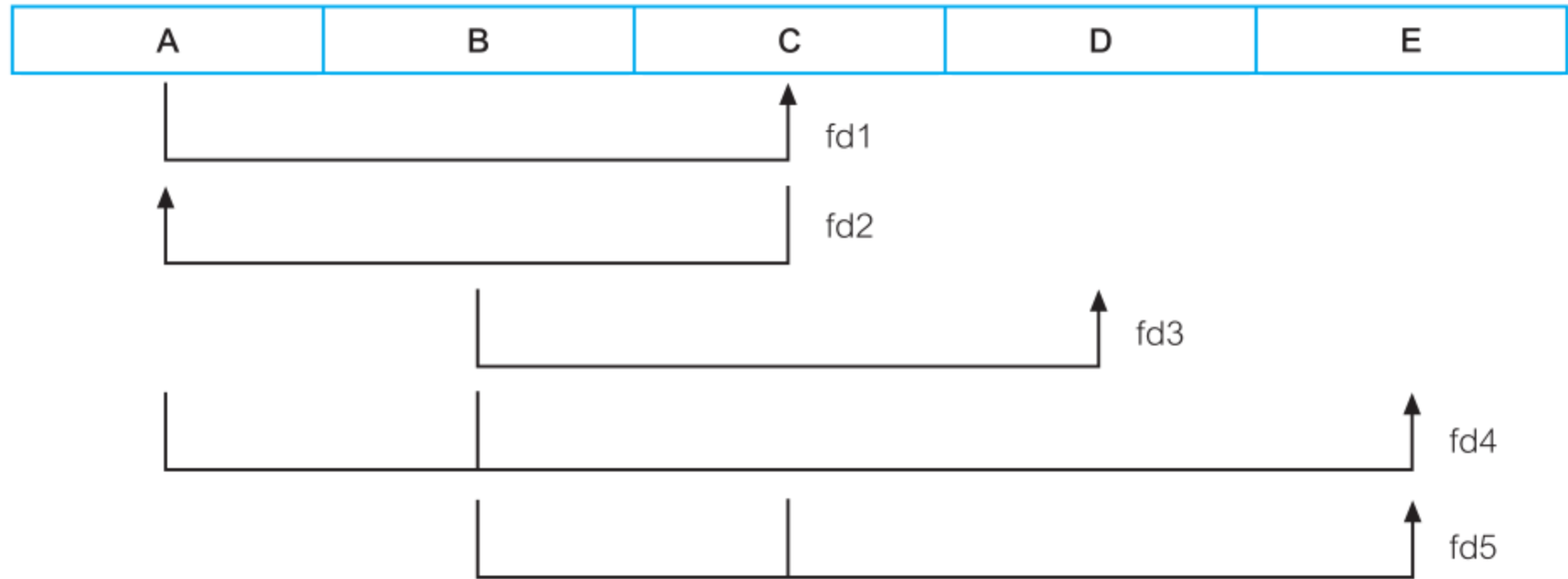
- ❑ We now consider the situation where functional dependencies are to be identified in the absence of appropriate information about the meaning of attributes and their relationships.
- ❑ In this case, it may be possible to identify functional dependencies if sample data is available that is a true representation of ***all*** possible data values that the database may hold.
- ❑ Consider the data for attributes denoted A, B, C, D, and E in a **Sample** relation.

Sample Relation

A	B	C	D	E
a	b	z	w	q
e	b	r	w	p
a	d	z	w	t
e	d	r	w	q
a	f	z	s	t
e	f	r	s	t



Sample Relation



In summary, we describe the function dependencies between attributes A to E in the Sample relation as follows:

- A $\rightarrow$ C (fd1)
- C $\rightarrow$ A (fd2)
- B $\rightarrow$ D (fd3)
- A, B $\rightarrow$ E (fd4)
- B, C $\rightarrow$ E (fd5)

Identifying the Primary Key

- ❑ The main purpose of identifying a set of functional dependencies for a relation is to specify the set of integrity constraints that must hold on a relation.
- ❑ An important integrity constraint to consider first is the identification of candidate keys, one of which is selected to be the primary key for the relation.
- ❑ We demonstrate the identification of a primary key for a given relation using the previous two examples (the **StaffBranch** and the **Sample** relations).

The Primary Key for the StaffBranch Relation

StaffBranch

staffNo	sName	position	salary	branchNo	bAddress
SL21	John White	Manager	30000	B005	22 Deer Rd, London
SG37	Ann Beech	Assistant	12000	B003	163 Main St, Glasgow
SG14	David Ford	Supervisor	18000	B003	163 Main St, Glasgow
SA9	Mary Howe	Assistant	9000	B007	16 Argyll St, Aberdeen
SG5	Susan Brand	Manager	24000	B003	163 Main St, Glasgow
SL41	Julie Lee	Assistant	9000	B005	22 Deer Rd, London

- staffNo ® sName, position, salary, branchNo, bAddress
- branchNo ® bAddress
- bAddress ® branchNo
- branchNo, position ® salary
- bAddress, position ® salary

The Primary Key for the **StaffBranch** Relation

- ❑ The **StaffBranch** relation has five functional dependencies:
 - $\text{staffNo} \twoheadrightarrow \text{sName, position, salary, branchNo, bAddress}$
 - $\text{branchNo} \twoheadrightarrow \text{bAddress}$
 - $\text{bAddress} \twoheadrightarrow \text{branchNo}$
 - $\text{branchNo, position} \twoheadrightarrow \text{salary}$
 - $\text{bAddress, position} \twoheadrightarrow \text{salary}$
- ❑ To identify the candidate key(s) for the **StaffBranch** relation, we must identify the attribute (or group of attributes) that uniquely identifies each tuple in this relation.
- ❑ All attributes that are not part of the candidate key should be functionally dependent on the key.
- ❑ The only candidate key of the **StaffBranch** relation, and therefore the primary key, is **staffNo**, as all other attributes of the relation are functionally dependent on **staffNo**.

The Primary Key for the **Sample** Relation

$A \rightarrow C$
 $C \rightarrow A$
 $B \rightarrow D$
 $A, B \rightarrow E$
 $B, C \rightarrow E$

- ❑ The **Sample** relation has five functional dependencies:
- ❑ The only determinants that determine all the other attributes of the relation are **(A,B)** and **(B,C)**.
- ❑ In the case of **(A,B)**, **A** functionally determines **C**, **B** functionally determines **D**, and **(A,B)** functionally determines **E**.
- ❑ In other words, the attributes that make up the determinant **(A,B)** can determine all the other attributes in the relation either separately as **A** or **B** or together as **(A,B)**.

The Process of Normalization

- ❑ The technique involves a series of rules that can be used to test individual relations so that a database can be normalized to any degree.
- ❑ Three normal forms were initially proposed called First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF).
- ❑ Normalization is often executed as a series of steps. Each step corresponds to a specific normal form that has known properties.
- ❑ As normalization proceeds, the relations become progressively more restricted (stronger) in format and also less vulnerable to update anomalies.

First Normal Form (1NF)

- ❑ **Unnormalized Form (UNF):** A table that contains one or more repeating groups - (attribute or group of attributes that can have multiple values associated with a single occurrence in the table).
- ❑ **First Normal Form (1NF):** A relation in which the intersection of each row and column contains one and only one value.
- ❑ The process of normalization begins by first transferring the data from the source (for example, a standard data entry form) into table format with rows and columns. In this format, the table is in unnormalized Form and is referred to as an *unnormalized table*.
- ❑ To transform the *unnormalized table* to (1NF), we identify and remove repeating groups within the table.

DreamHome Lease

DreamHome Lease

DreamHome Lease

DreamHome Lease

Client Number CR76
(Enter if known)

Full Name John Kay
(Please print)

Property Number PG4

Property Address
6 Lawrence St, Glasgow

Monthly Rent 350

Rent Start 01/07/12

Rent Finish 31/08/13

Owner Number C040
(Enter if known)

Full Name Tina Murphy
(Please print)

First Normal Form (1NF)

- ❑ Sample data is taken from leases for two different clients and is transformed into an *unnormalized table* with rows and columns.

ClientRental

clientNo	cName	propertyNo	pAddress	rentStart	rentFinish	rent	ownerNo	oName
CR76	John Kay	PG4	6 Lawrence St, Glasgow	1-Jul-12	31-Aug-13	350	CO40	Tina Murphy
		PG16	5 Novar Dr, Glasgow	1-Sep-13	1-Sep-14	450	CO93	Tony Shaw
CR56	Aline Stewart	PG4	6 Lawrence St, Glasgow	1-Sep-11	10-June-12	350	CO40	Tina Murphy
		PG36	2 Manor Rd, Glasgow	10-Oct-12	1-Dec-13	375	CO93	Tony Shaw
		PG16	5 Novar Dr, Glasgow	1-Nov-14	10-Aug-15	450	CO93	Tony Shaw

- ❑ Repeating Group is:
(propertyNo, pAddress, rentStart, rentFinish, rent, ownerNo, oName)
- ❑ To transform an *unnormalized table* into 1NF, we remove the repeating group by entering the appropriate client data into each row.

ClientRental

(UNF)

clientNo	cName	propertyNo	pAddress	rentStart	rentFinish	rent	ownerNo	oName
CR76	John Kay	PG4	6 Lawrence St, Glasgow	1-Jul-12	31-Aug-13	350	CO40	Tina Murphy
		PG16	5 Novar Dr, Glasgow	1-Sep-13	1-Sep-14	450	CO93	Tony Shaw
CR56	Aline Stewart	PG4	6 Lawrence St, Glasgow	1-Sep-11	10-June-12	350	CO40	Tina Murphy
		PG36	2 Manor Rd, Glasgow	10-Oct-12	1-Dec-13	375	CO93	Tony Shaw
		PG16	5 Novar Dr, Glasgow	1-Nov-14	10-Aug-15	450	CO93	Tony Shaw

ClientRental

(1NF)

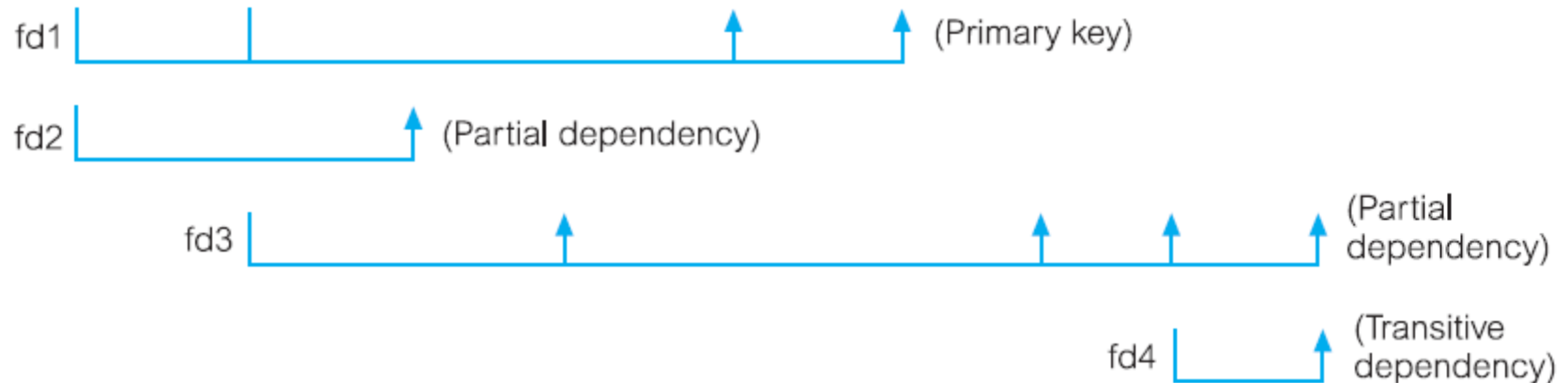
clientNo	propertyNo	cName	pAddress	rentStart	rentFinish	rent	ownerNo	oName
CR76	PG4	John Kay	6 Lawrence St, Glasgow	1-Jul-12	31-Aug-13	350	CO40	Tina Murphy
CR76	PG16	John Kay	5 Novar Dr, Glasgow	1-Sep-13	1-Sep-14	450	CO93	Tony Shaw
CR56	PG4	Aline Stewart	6 Lawrence St, Glasgow	1-Sep-11	10-Jun-12	350	CO40	Tina Murphy
CR56	PG36	Aline Stewart	2 Manor Rd, Glasgow	10-Oct-12	1-Dec-13	375	CO93	Tony Shaw
CR56	PG16	Aline Stewart	5 Novar Dr, Glasgow	1-Nov-14	10-Aug-15	450	CO93	Tony Shaw

The Process of Normalization

- For this worked example, we assume that a client rents a given property only once.

ClientRental

clientNo	propertyNo	cName	pAddress	rentStart	rentFinish	rent	ownerNo	oName
----------	------------	-------	----------	-----------	------------	------	---------	-------



The Process of Normalization

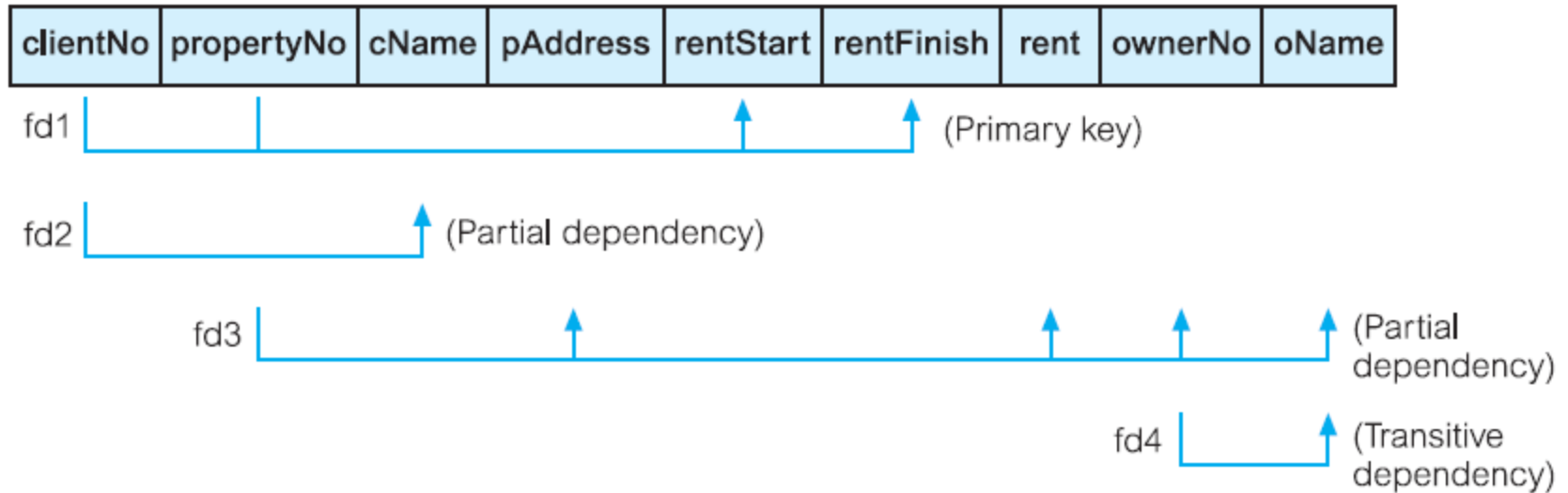
- ❑ The primary key for the **ClientRental** relation is (clientNo, propertyNo).
- ❑ The **ClientRental** relation is in 1NF, as there is a single value at the intersection of each row and column.
- ❑ The relation contains data describing clients, property rented, and property owners, which is repeated several times. As a result, the **ClientRental** relation contains significant data redundancy.
- ❑ If implemented, the 1NF relation would be subject to the update anomalies. To remove some of these, we must transform the relation into second normal form (2NF) .

Second Normal Form (2NF)

- ❑ **Second Normal Form (2NF):** A relation that is in first normal form and every non-primary-key attribute is fully functionally dependent on the primary key.
- ❑ The normalization of 1NF relations to 2NF involves the removal of partial dependencies.
- ❑ If a partial dependency exists, we remove the partially dependent attribute(s) from the relation by placing them in a new relation along with a copy of their determinant.

Second Normal Form (2NF)

- As shown, the **ClientRental** relation has the following functional dependencies:



- We begin by testing whether the **ClientRental** relation is in 2NF by identifying the presence of any partial dependencies on the primary key.

Second Normal Form (2NF)

- ❑ The identification of partial dependencies within the **ClientRental** relation indicates that the relation is not in 2NF.
- ❑ To transform the **ClientRental** relation into 2NF requires the creation of new relations so that the non-primary-key attributes are removed along with a copy of the part of the primary key on which they are fully functionally dependent.
- ❑ This results in the creation of three new relations that are in second normal form:

Client (clientNo, cName)

PropertyOwner (propertyNo, pAddress, rent, ownerNo, oName)

Rental (clientNo, propertyNo, rentStart, rentFinish)

First Normal Form (1NF)

ClientRental

clientNo	propertyNo	cName	pAddress	rentStart	rentFinish	rent	ownerNo	oName
CR76	PG4	John Kay	6 Lawrence St, Glasgow	1-Jul-12	31-Aug-13	350	CO40	Tina Murphy
CR76	PG16	John Kay	5 Novar Dr, Glasgow	1-Sep-13	1-Sep-14	450	CO93	Tony Shaw
CR56	PG4	Aline Stewart	6 Lawrence St, Glasgow	1-Sep-11	10-Jun-12	350	CO40	Tina Murphy
CR56	PG36	Aline Stewart	2 Manor Rd, Glasgow	10-Oct-12	1-Dec-13	375	CO93	Tony Shaw
CR56	PG16	Aline Stewart	5 Novar Dr, Glasgow	1-Nov-14	10-Aug-15	450	CO93	Tony Shaw

Second Normal Form (2NF)

Client

clientNo	cName
CR76	John Kay
CR56	Aline Stewart

PropertyOwner

propertyNo	pAddress	rent	ownerNo	oName
PG4	6 Lawrence St, Glasgow	350	CO40	Tina Murphy
PG16	5 Novar Dr, Glasgow	450	CO93	Tony Shaw
PG36	2 Manor Rd, Glasgow	375	CO93	Tony Shaw

Rental

clientNo	propertyNo	rentStart	rentFinish
CR76	PG4	1-Jul-12	31-Aug-13
CR76	PG16	1-Sep-13	1-Sep-14
CR56	PG4	1-Sep-11	10-Jun-12
CR56	PG36	10-Oct-12	1-Dec-13
CR56	PG16	1-Nov-14	10-Aug-15

Third Normal Form (3NF)

- ❑ **Third Normal Form (3NF):** A relation that is in first and second normal form and in which no attribute is transitively dependent on the primary key.
- ❑ The normalization of 2NF relations to 3NF involves the removal of transitive dependencies.
- ❑ If a transitive dependency exists, we remove the transitively dependent attribute(s) from the relation by placing the attribute(s) in a new relation along with a copy of the determinant.

- All the non-primary-key attributes within the **PropertyOwner** relation are functionally dependent on the primary key, with the exception of **oName**, which is transitively dependent on **ownerNo**.

propertyNo	pAddress	rent	ownerNo	oName
PG4	6 Lawrence St, Glasgow	350	CO40	Tina Murphy
PG16	5 Novar Dr, Glasgow	450	CO93	Tony Shaw
PG36	2 Manor Rd, Glasgow	375	CO93	Tony Shaw

fd4  (Transitive dependency)

- To transform the **PropertyOwner** relation into 3NF, we must first remove this transitive dependency by creating two new relations called **PropertyForRent** and **Owner**.
- The new relations have the following forms:
PropertyForRent (propertyNo, pAddress, rent, ownerNo)
Owner (ownerNo, oName)

Third Normal Form (3NF)

- Third normal form relations derived from the **PropertyOwner** relation.

PropertyForRent

propertyNo	pAddress	rent	ownerNo
PG4	6 Lawrence St, Glasgow	350	CO40
PG16	5 Novar Dr, Glasgow	450	CO93
PG36	2 Manor Rd, Glasgow	375	CO93

Owner

ownerNo	oName
CO40	Tina Murphy
CO93	Tony Shaw

- The **ClientRental** relation has been transformed by the process of normalization into four relations in 3NF.

Client (clientNo, cName)

PropertyForRent (propertyNo, pAddress, rent, ownerNo)

Owner (ownerNo, oName)

Rental (clientNo, propertyNo, rentStart, rentFinish)

Third Normal Form (3NF)

Client

clientNo	cName
CR76	John Kay
CR56	Aline Stewart

Rental

clientNo	propertyNo	rentStart	rentFinish
CR76	PG4	1-Jul-12	31-Aug-13
CR76	PG16	1-Sep-13	1-Sep-14
CR56	PG4	1-Sep-11	10-Jun-12
CR56	PG36	10-Oct-12	1-Dec-13
CR56	PG16	1-Nov-14	10-Aug-15

PropertyForRent

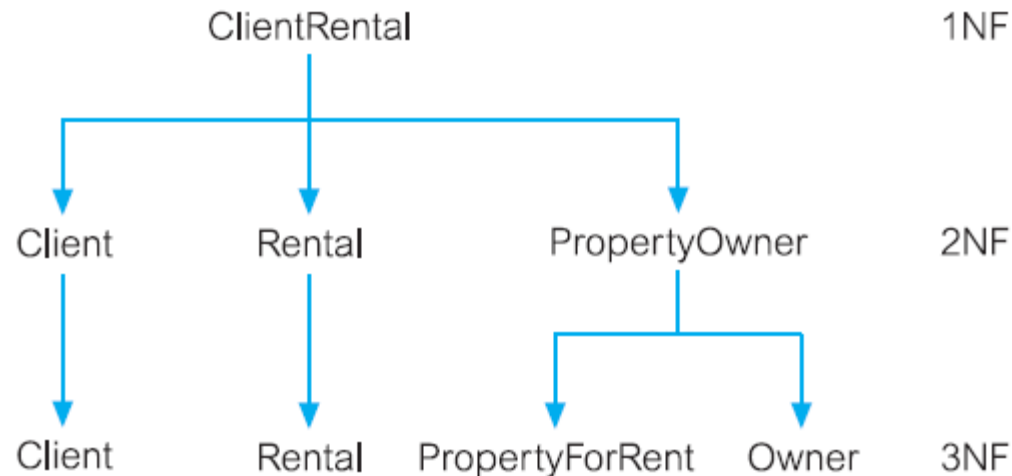
propertyNo	pAddress	rent	ownerNo
PG4	6 Lawrence St, Glasgow	350	CO40
PG16	5 Novar Dr, Glasgow	450	CO93
PG36	2 Manor Rd, Glasgow	375	CO93

Owner

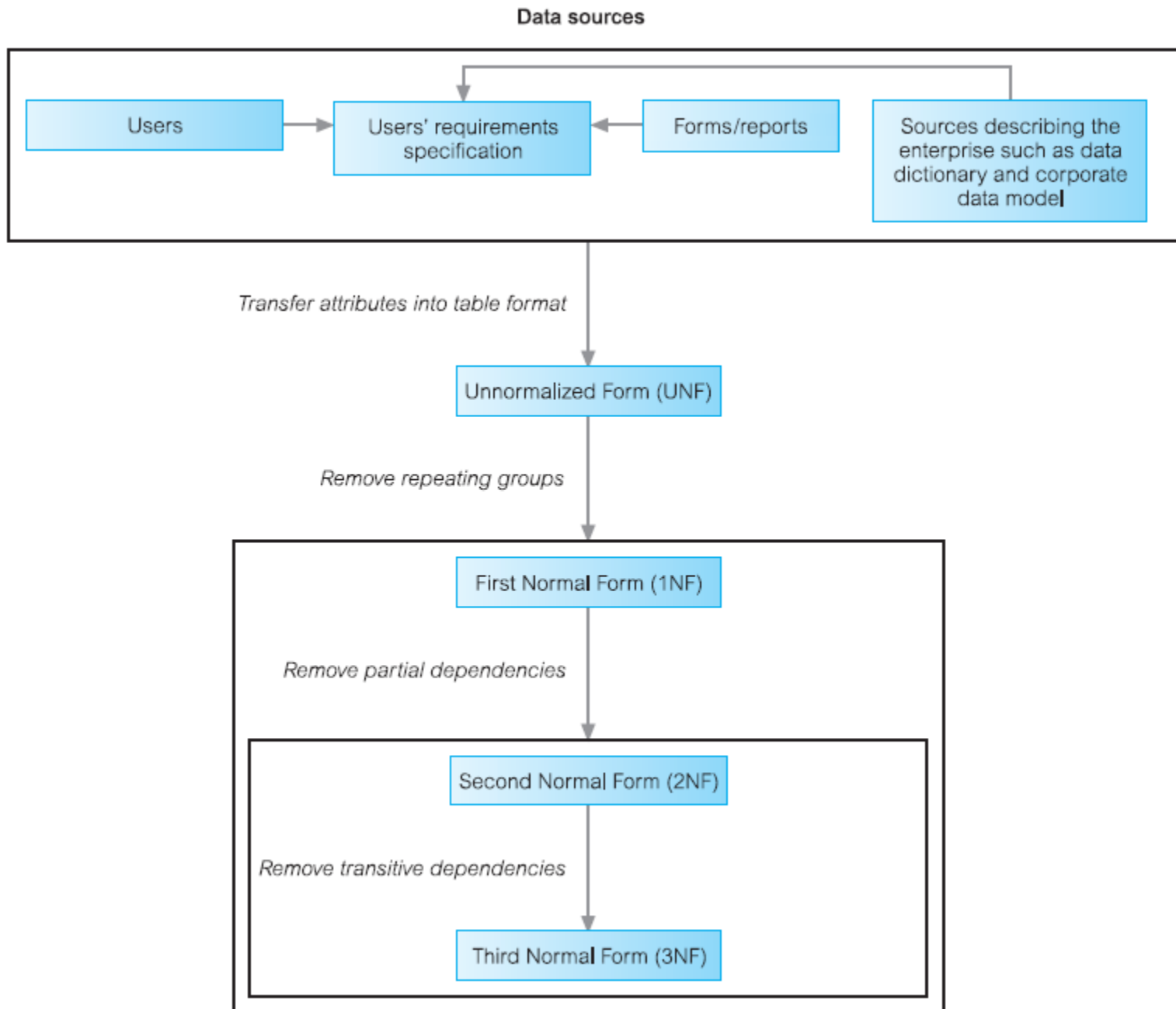
ownerNo	oName
CO40	Tina Murphy
CO93	Tony Shaw

The Process of Normalization

- The normalization process has decomposed the original **ClientRental** relation using a series of relational algebra projections. This results in a lossless-join decomposition, which is reversible using the natural join operation.



Diagrammatic illustration of the normalization process



Case Study

- ❑ Examine the Patient Medication Form for the *Wellmeadows* Hospital case study then produce a set of well-designed 3NF relations.

Wellmeadows Hospital Patient Medication Form

Patient Number: P10034

Full Name: Robert MacDonald

Ward Number: Ward 11

Bed Number: 84

Ward Name: Orthopaedic

Drug Number	Name	Description	Dosage	Units per Day	Start Date	Finish Date
10223	Morphine	Pain Killer	10mg/ml	50	24/03/13	24/04/13
10334	Tetracycline	Antibiotic	0.5mg/ml	10	24/03/13	17/04/13
10223	Morphine	Pain Killer	10mg/ml	10	25/04/13	02/05/13

Wellmeadows Hospital Patient Medication Form

Patient Number: P10034

Full Name: Robert MacDonald

Ward Number: Ward 11

Bed Number: 84

Ward Name: Orthopaedic

Drug Number	Name	Description	Dosage	Units per Day	Start Date	Finish Date
10223	Morphine	Pain Killer	10mg/ml	50	24/03/13	24/04/13
10334	Tetracycline	Antibiotic	0.5mg/ml	10	24/03/13	17/04/13
10223	Morphine	Pain Killer	10mg/ml	10	25/04/13	02/05/13



PatientDrugMedication

patientNo	fullName	bedNo	wardNo	wardName	drugNo	name	description	dosage	unitsDay	startDate	finishDate
P10034	Robert MacDonald	84	Ward 11	Orthopaedic	10223	Morphine	Pain Killer	10mg/ml	50	24/03/13	24/04/13
					10334	Tetracycline	Antibiotic	0.5mg/ml	10	24/03/13	17/04/13
					10223	Morphine	Pain Killer	10mg/ml	10	25/04/13	02/05/13
P10041	John Walker	106	Ward 13	Ophthalmic	10223	Morphine	Pain Killer	10mg/ml	10	13/05/13	26/05/13
					20366	Ofloxacin	Antibiotic	0.5mg/ml	10	13/05/13	30/05/13
P10051	Alexandra Garcia	86	Ward 11	Orthopaedic	10334	Tetracycline	Antibiotic	0.5mg/ml	5	13/05/13	18/05/13

First Normal Form (1NF)

□ UNF

PatientDrugMedication

patientNo	fullName	bedNo	wardNo	wardName	drugNo	name	description	dosage	unitsDay	startDate	finishDate
P10034	Robert MacDonald	84	Ward 11	Orthopaedic	10223	Morphine	Pain Killer	10mg/ml	50	24/03/13	24/04/13
					10334	Tetracycline	Antibiotic	0.5mg/ml	10	24/03/13	17/04/13
					10223	Morphine	Pain Killer	10mg/ml	10	25/04/13	02/05/13
P10041	John Walker	106	Ward 13	Ophthalmic	10223	Morphine	Pain Killer	10mg/ml	10	13/05/13	26/05/13
					20366	Ofloxacin	Antibiotic	0.5mg/ml	10	13/05/13	30/05/13
P10051	Alexandra Garcia	86	Ward 11	Orthopaedic	10334	Tetracycline	Antibiotic	0.5mg/ml	5	13/05/13	18/05/13

□ 1NF

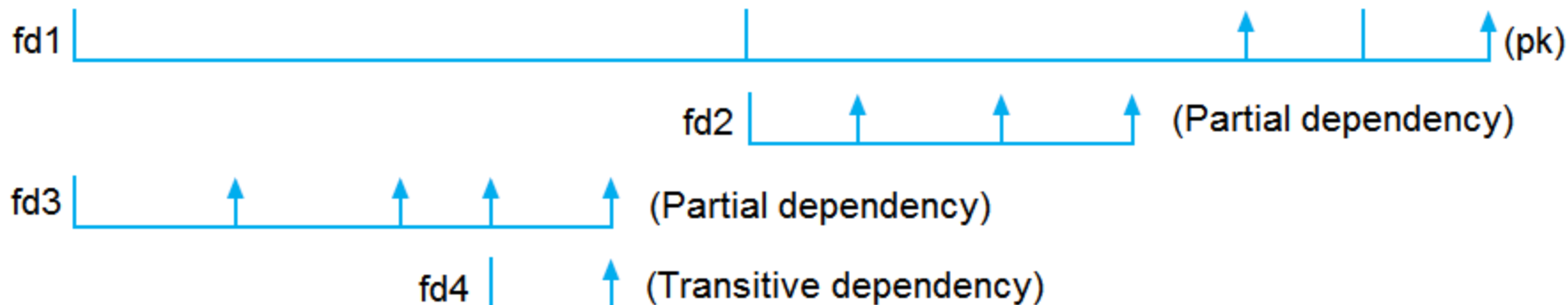
PatientDrugMedication

patientNo	fullName	bedNo	wardNo	wardName	drugNo	name	description	dosage	unitsDay	startDate	finishDate
P10034	Robert MacDonald	84	Ward 11	Orthopaedic	10223	Morphine	Pain Killer	10mg/ml	50	24/03/13	24/04/13
P10034	Robert MacDonald	84	Ward 11	Orthopaedic	10334	Tetracycline	Antibiotic	0.5mg/ml	10	24/03/13	17/04/13
P10034	Robert MacDonald	84	Ward 11	Orthopaedic	10223	Morphine	Pain Killer	10mg/ml	10	25/04/13	02/05/13
P10041	John Walker	106	Ward 13	Ophthalmic	10223	Morphine	Pain Killer	10mg/ml	10	13/05/13	26/05/13
P10041	John Walker	106	Ward 13	Ophthalmic	20366	Ofloxacin	Antibiotic	0.5mg/ml	10	13/05/13	30/05/13
P10051	Alexandra Garcia	86	Ward 11	Orthopaedic	10334	Tetracycline	Antibiotic	0.5mg/ml	5	13/05/13	18/05/13

(1NF)

PatientDrugMedication

patientNo	fullName	bedNo	wardNo	wardName	drugNo	name	description	dosage	unitsDay	startDate	finishDate
P10034	Robert MacDonald	84	Ward 11	Orthopaedic	10223	Morphine	Pain Killer	10mg/ml	50	24/03/13	24/04/13
P10034	Robert MacDonald	84	Ward 11	Orthopaedic	10334	Tetracycline	Antibiotic	0.5mg/ml	10	24/03/13	17/04/13
P10034	Robert MacDonald	84	Ward 11	Orthopaedic	10223	Morphine	Pain Killer	10mg/ml	10	25/04/13	02/05/13
P10041	John Walker	106	Ward 13	Ophthalmic	10223	Morphine	Pain Killer	10mg/ml	10	13/05/13	26/05/13
P10041	John Walker	106	Ward 13	Ophthalmic	20366	Ofloxacin	Antibiotic	0.5mg/ml	10	13/05/13	30/05/13
P10051	Alexandra Garcia	86	Ward 11	Orthopaedic	10334	Tetracycline	Antibiotic	0.5mg/ml	5	13/05/13	18/05/13



(2NF)

Medication

patientNo	drugNo	unitsDay	startDate	finishDate
P10034	10223	50	24/03/13	24/04/13
P10034	10334	10	24/03/13	17/04/13
P10034	10223	10	25/04/13	02/05/13
P10041	10223	10	13/05/13	26/05/13
P10041	20366	10	13/05/13	30/05/13
P10051	10334	5	13/05/13	18/05/13

Drug

drugNo	name	description	dosage
10223	Morphine	Pain Killer	10mg/ml
10334	Tetracycline	Antibiotic	0.5mg/ml
20366	Ofloxacin	Antibiotic	0.5mg/ml

PatientWard

patientNo	fullName	bedNo	wardNo	wardName
P10034	Robert MacDonald	84	Ward 11	Orthopaedic
P10041	John Walker	106	Ward 13	Ophthalmic
P10051	Alexandra Garcia	86	Ward 11	Orthopaedic

(2NF)

Medication

patientNo	drugNo	unitsDay	startDate	finishDate
P10034	10223	50	24/03/13	24/04/13
P10034	10334	10	24/03/13	17/04/13
P10034	10223	10	25/04/13	02/05/13
P10041	10223	10	13/05/13	26/05/13
P10041	20366	10	13/05/13	30/05/13
P10051	10334	5	13/05/13	18/05/13

Drug

drugNo	name	description	dosage
10223	Morphine	Pain Killer	10mg/ml
10334	Tetracycline	Antibiotic	0.5mg/ml
20366	Ofloxacin	Antibiotic	0.5mg/ml

PatientWard

patientNo	fullName	bedNo	wardNo	wardName
P10034	Robert MacDonald	84	Ward 11	Orthopaedic
P10041	John Walker	106	Ward 13	Ophthalmic
P10051	Alexandra Garcia	86	Ward 11	Orthopaedic


(Transitive dependency)

(3NF)

Medication

patientNo	drugNo	unitsDay	startDate	finishDate
P10034	10223	50	24/03/13	24/04/13
P10034	10334	10	24/03/13	17/04/13
P10034	10223	10	25/04/13	02/05/13
P10041	10223	10	13/05/13	26/05/13
P10041	20366	10	13/05/13	30/05/13
P10051	10334	5	13/05/13	18/05/13

Drug

drugNo	name	description	dosage
10223	Morphine	Pain Killer	10mg/ml
10334	Tetracycline	Antibiotic	0.5mg/ml
20366	Ofloxacin	Antibiotic	0.5mg/ml

Patient

patientNo	fullName	bedNo	wardNo
P10034	Robert MacDonald	84	Ward 11
P10041	John Walker	106	Ward 13
P10051	Alexandra Garcia	86	Ward 11

Ward

wardNo	wardName
Ward 11	Orthopaedic
Ward 13	Ophthalmic

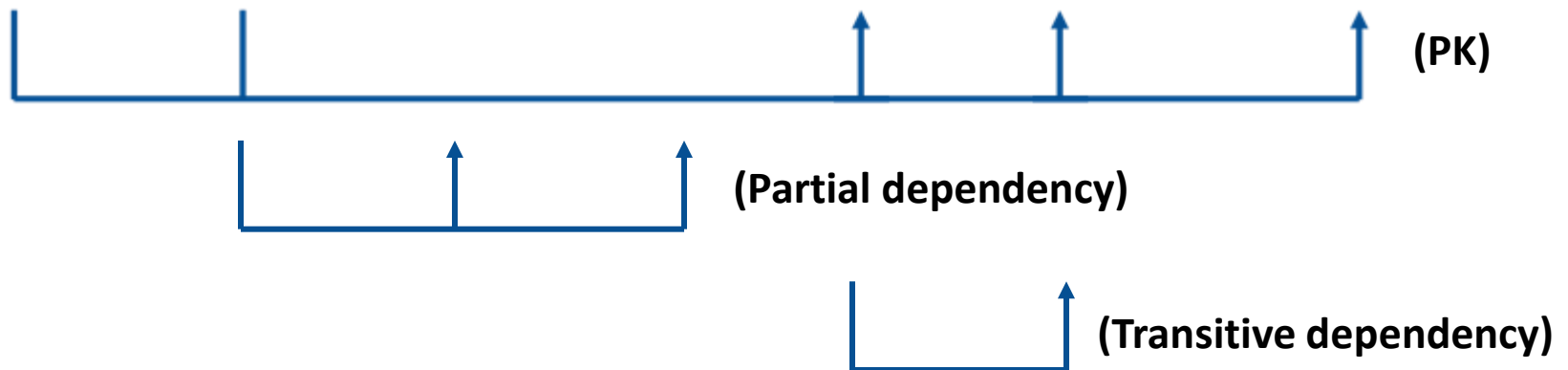
Exercise 1

- ❑ A company provides a taxi service to clients.
 - ❑ The below table displays some details of client bookings for taxis.
 - ❑ Assume that a taxi driver is assigned to a single taxi, but a taxi can be assigned to one or more drivers.
1. Identify the functional dependencies that exist between the columns of the table and identify the primary key.
 2. Illustrate the process of normalizing the table to 3NF.

JobDate Time	driverID	driver Name	taxiID	clientID	clientName	jobPickUpAddress
25/07/14 10.00	D1	Joe Bull	T1	C1	Anne Woo	1 Storrie Rd, Paisley
29/07/14 10.00	D1	Joe Bull	T1	C1	Anne Woo	1 Storrie Rd, Paisley
30/07/14 11.00	D2	Tom Win	T2	C1	Anne Woo	3 High Street, Paisley
2/08/14 13.00	D3	Jim Jones	T3	C2	Mark Tin	1A Lady Lane, Paisley
2/08/14 13.00	D4	Steven Win	T1	C3	John Seal	22 Red Road, Paisley
25/08/14 10.00	D2	Tom Win	T2	C4	Karen Bow	17 High Street, Paisley

Exercise 1 - Solution (Functional Dependencies)

JobDate Time	driverID	driver Name	taxiID	clientID	clientName	jobPickUpAddress
25/07/14 10.00	D1	Joe Bull	T1	C1	Anne Woo	1 Storrie Rd, Paisley
29/07/14 10.00	D1	Joe Bull	T1	C1	Anne Woo	1 Storrie Rd, Paisley
30/07/14 11.00	D2	Tom Win	T2	C1	Anne Woo	3 High Street, Paisley
2/08/14 13.00	D3	Jim Jones	T3	C2	Mark Tin	1A Lady Lane, Paisley
2/08/14 13.00	D4	Steven Win	T1	C3	John Seal	22 Red Road, Paisley
25/08/14 10.00	D2	Tom Win	T2	C4	Karen Bow	17 High Street, Paisley



Primary key: JobDate Time, driverID

Exercise 1 - Solution (Normalized Tables)

JobDateTime	driverID	clientID	jobPickUpAddress
25/07/14 10.00	D1	C1	1 Storrier Rd, Paisley
29/07/14 10.00	D1	C1	1 Storrier Rd, Paisley
30/07/14 11.00	D2	C1	3 High Street, Paisley
2/08/14 13.00	D3	C2	1A Lady Lane, Paisley
2/08/14 13.00	D4	C3	22 Red Road, Paisley
25/08/14 10.00	D2	C4	17 High Street, Paisley

driverID	driverName	taxiID
D1	Joe Bull	T1
D2	Tom Win	T2
D3	Jim Jones	T3
D4	Steven Win	T1

clientID	clientName
C1	Anne Woo
C2	Mark Tin
C3	John Seal
C4	Karen Bow

Exercise 2

- ❑ Details of leases held by students for places in university flats are shown in the below table.
 - ❑ A room number (**RoomNo**) uniquely identifies each single room in all flats and is used when leasing a room to a student.
1. Identify the functional dependencies that exist between the columns of the table and identify the primary key.
 2. Illustrate the process of normalizing the table to 3NF.

StudentID	RoomNo	fName	lName	startDate	finishDate	flatNo	flatAddress
B017706	78	Jane	Watt	01/09/2010	30/06/2011	F56	34 High Street, Paisley
B017706	88	Jane	Watt	01/09/2011	30/06/2012	F78	111 Storrie Road, Paisley
B013399	89	Tom	Jones	01/09/2011	30/06/2012	F78	111 Storrie Road, Paisley
B012124	102	Karen	Black	01/09/2011	30/06/2012	F79	120 Lady Lane, Paisley
B034511	88	Steven	Smith	01/09/2012	30/06/2013	F78	111 Storrie Road, Paisley
B013399	78	Tom	Jones	01/09/2012	30/06/2013	F56	34 High Street, Paisley

Exercise 2 - Solution

StudentID	RoomNo	startDate	finishDate
B017706	78	01/09/2010	30/06/2011
B017706	88	01/09/2011	30/06/2012
B013399	89	01/09/2011	30/06/2012
B012124	102	01/09/2011	30/06/2012
B034511	88	01/09/2012	30/06/2013
B013399	78	01/09/2012	30/06/2013

Primary key: StudentID, startDate

StudentID	fName	lName
B017706	Jane	Watt
B013399	Tom	Jones
B012124	Karen	Black
B034511	Steven	Smith

RoomNo	flatNo
78	F56
88	F78
89	F78
102	F79

flatNo	flat Address
F56	34 High Street, Paisley
F78	111 Storrie Road, Paisley
F79	120 Lady Lane, Paisley

Exercise 3

□ Consider a relation $R=(A, B, C, D, E, F, G)$ that satisfies the following FDs.

$$A \rightarrow B$$

$$A, C \rightarrow D, E$$

$$D \rightarrow F$$

$$E \rightarrow G$$

1. Identify the primary key of the relation R .
2. Normalize the relation R to the $2NF$.
3. Normalize the resulting relations to the $3NF$.

Exercise 3 - Solution

1. Primary key: A, C
2. $2NF$:
 (A, B)
 (A, C, D, E, F, G)
3. $3NF$:
 (A, B)
 (D, F)
 (E, G)
 (A, C, D, E)